Tool Segmentation and Contact-Region Estimation in Endoscopic Images

Letizia Sorriento
Federico De Angelis
Claudia Julia Istoc
Martina Leggiero

Abstract

This report presents a computer-vision pipeline for detecting surgical tools in endoscopic images and estimating the region in which a tool is likely to interact with tissue. The system combines two independently trained segmentation models—**YOLO** instance segmentation and a **multiclass U-Net**—with monocular depth estimation based on **Depth Anything V2**. The models are trained on architecture-specific representations of the same annotated frames: YOLO uses polygon labels, whereas U-Net uses raster semantic masks. A depth-based fusion stage restricts the predicted tool region to a selected quantile of pixels according to their relative depth. The resulting information defines a **Region Of Interest** (**ROI**), which is passed to a fine-tuned ResNet classifier (**ROIClassifier**) to predict whether a **Tool-Tissue Interaction** (**TTI**) has occurred.

The approach is evaluated using segmentation and classification metrics, including IoU, Dice, precision, recall, accuracy, macro-F1, and confusion matrices. The dataset contains 2,880 training, 760 validation, and 182 test images. On the validation set, YOLO26n-seg achieved a box mAP@50 of 0.455 and a mask mAP@50 of 0.426, with performance varying substantially across classes due to the uneven distribution. The U-Net experiments demonstrate progressive improvement during training, although foreground segmentation remains more difficult than overall pixel accuracy because of strong class imbalance.

The controlled end-to-end comparison shows that YOLO11 provides a better overall trade-off than YOLO26, while YOLO26 achieves higher precision and specificity. Depth fusion generates interpretable contact-region hypotheses, although these should be evaluated separately from tool segmentation and calibrated against ground-truth interaction annotations. All experiments were conducted using Ultralytics 8.4.136, PyTorch 2.11.0 with CUDA 12.8 acceleration, and an NVIDIA GeForce RTX 5060 Laptop GPU.

1 Introduction

Endoscopic surgery requires a robotic or computer-assisted system to interpret complex visual scenes containing instruments, tissue, blood, specular reflections, smoke, and frequent occlusions. A fundamental perception task is the localization of surgical tools. Tool segmentation provides a pixel-level description of the instrument, which can support tracking, scene understanding, interaction analysis, and downstream robotic control. However, a tool mask alone does not identify which portion of the instrument is closest to or most likely to be interacting with tissue. This motivates a second stage that combines segmentation with monocular depth estimation.

The Medical Robotics project investigates two complementary segmentation pipelines. The first uses YOLO instance segmentation, selected for its integrated detection and mask-prediction capabilities. The second uses a U-Net architecture adapted to multiclass semantic segmentation. Both models operate on the same annotated endoscopic frames, but the data are exported in different formats. The YOLO dataset contains images and polygon labels for tool and tissue–tool interaction annotations. The U-Net dataset contains images and raster masks, including

semantic tool masks and interaction-related masks. The two representations share the same train/validation/test split, ensuring that both models are evaluated on corresponding data partitions.

Depth Anything V2 is used as an additional, pretrained monocular depth estimator. Its output is not treated as metric distance. Instead, the depth map is normalized within each image, and depth values inside the predicted tool region are ranked. A quantile-based fusion rule selects either the lower or higher portion of the depth distribution. The result is filtered with a median operation and connected-component analysis to remove small regions. This produces a contact-mask hypothesis rather than a directly supervised contact prediction.

The main contributions of the project are therefore:

- a consistent dual-format dataset preparation procedure;
- two alternative tool-segmentation models;
- a multiclass U-Net training strategy combining weighted cross-entropy and Dice loss;
- a depth-based contact-region extraction stage;
- an end-to-end evaluation and visualization framework.

2 Methodology

2.1 Dataset preparation, organization and consistency

The original dataset consisted of surgical video clips and JSON annotations describing surgical instruments and Tool–Tissue Interactions (TTIs) through polygon coordinates and class information.

A preprocessing pipeline (`create_dataset.py`) converted the data into two representations: YOLO-Seg labels and U-Net semantic masks. For YOLO, classes 0–11 represent surgical instruments and classes 12–20 represent TTI categories. For U-Net, the same frames were converted into raster masks for tools and TTIs.

The original 3,938 annotated frames were filtered by removing 106 TTI-only frames and 10 frames with invalid indices, resulting in 3,822 valid frames: 2,880 training, 760 validation, and 182 test (Table 1).

Table 1: Dataset preprocessing results.

Split	Annotated	TTI-only	Invalid	Final
Train	2,989	106	3	2,880
Validation	767	0	7	760
Test	182	0	0	182
Total	3,938	106	10	3,822

The YOLO and U-Net representations share the same train/validation/test frames. `labels.ipynb` merged tool and TTI annotations for YOLO, while `dataset_summary.py` verified consistency and analyzed class distributions.

U-Net uses 512×512 images, batch size 2, AdamW with a learning rate of 10^{-4}, weight decay of 10^{-5}, and approximately 1.93M parameters. YOLO11 and YOLO26 use 640×640 images, batch size 16, and are trained for up to 150 epochs with an early-stopping patience of 100. Both models are evaluated on the same data partitions. Their outputs are later used as the segmentation components of the corresponding end-to-end baseline pipelines.

2.2 Multiclass U-Net

```
project_directory/
|-- program(yolo+unet).ipynb
```

The U-Net follows an encoder–decoder structure with skip connections. Each encoder and decoder block uses two convolutional layers, batch normalization, and ReLU activation. Max-pooling reduces spatial resolution in the encoder, while transposed convolutions restore it in the decoder. The final 1×1 convolution produces one logit channel per semantic class.

The dataset uses the same endoscopic frames and train/validation/test split as the YOLO pipeline, but annotations are stored as raster semantic masks rather than polygon labels. Each image is paired with a mask where each pixel encodes background, a surgical tool, or a tool–tissue interaction category.

Images are resized to 512×512 pixels using bilinear interpolation, while masks use nearest-neighbor interpolation to preserve discrete class values. During training, random horizontal and vertical flips are applied simultaneously to image–mask pairs; augmentation is disabled for validation and test.

The raw mask values are mapped to contiguous internal class indices. In the recorded configuration, the model predicts 21 classes (background, tool, and TTI classes). The predicted multiclass mask can be split into binary tool and TTI masks by grouping the corresponding class indices.

For an input tensor $\mathbb{R}^{3\times H\times W}$, the model produces logits $\mathbb{R}^{C\times H\times W}$. The predicted class at each pixel is:

$$\hat{y}(x,y) = \arg\max_{c\in\{0,\dots,C-1\}} z_c(x,y). \tag{1}$$

Softmax probabilities are also returned for confidence analysis and uncertainty inspection. Unlike YOLO26-seg, which predicts multiple object instances, U-Net assigns one semantic class to every pixel, producing a single multiclass segmentation map.

The training objective combines weighted cross-entropy and multiclass Dice loss:

$$\mathcal{L} = \lambda_{CE}\mathcal{L}_{CE} + \lambda_D\mathcal{L}_{Dice}, \tag{2}$$

with $\lambda_{CE} = \lambda_D = 0.5$. Weighted cross-entropy compensates for the uneven pixel distribution among classes, while Dice loss promotes spatial overlap between predicted and annotated regions, which is important because tools and interactions occupy only a small portion of the image.

Class weights are computed from the training-mask pixel distribution and clipped to avoid extreme values. A weighted random sampler increases the probability of selecting images containing under-represented foreground classes, reducing the influence of the dominant background class.

The model is trained for 50 epochs at 512×512 resolution, with batch size 2, learning rate 1×10^{-4}, and weight decay 1×10^{-5}. The best checkpoint is saved and used for validation, test inference, and visualization on the available CUDA device.

During inference, test images are preprocessed and passed through the trained U-Net. Output logits are converted to a multiclass prediction via pixel-wise arg max, then grouped into binary tool and TTI masks:

$$M_{\text{tool}}(x,y) = \begin{cases} 255, & \text{if } \hat{y}(x,y) \in \mathcal{C}_{\text{tool}}, \\ 0, & \text{otherwise}, \end{cases} \tag{3}$$

$$M_{\text{TTI}}(x,y) = \begin{cases} 255, & \text{if } \hat{y}(x,y) \in \mathcal{C}_{\text{TTI}}, \\ 0, & \text{otherwise}, \end{cases} \tag{4}$$

where $\mathcal{C}_{\text{tool}}$ and $\mathcal{C}_{\text{TTI}}$ are the sets of tool and interaction class indices. The multiclass mask, binary masks, and overlays are saved for the standalone evaluation of the U-Net predictions.

U-Net was evaluated as an alternative segmentation architecture to YOLO26. After training and validation, the U-Net results were compared with the YOLO26 segmentation metrics. Based on this comparison, YOLO26 demonstrated superior performance for the end-to-end TTI detection task and was selected as the segmentation component of the main pipeline. Consequently, U-Net was not integrated into the final end-to-end comparison and no depth-based processing was applied to its predictions.

2.3 YOLO26 segmentation pipeline

```
MR_project/
|-- program(yolo+unet).ipynb
```

YOLO26 is used as an instance-segmentation model for localizing and segmenting surgical tools and tool–tissue interaction categories. The model is trained and evaluated through the Ultralytics framework, which handles dataset loading, augmentation, optimization, validation, checkpoint selection, and result export.

The dataset configuration contains 21 semantic classes. Classes 0–11 correspond to surgical-tool categories, with `tool_10_UNUSED` retained only for consistency with the label mapping. Classes 12–20 correspond to interaction-related or tissue-related categories. Each annotated instance is represented by a class identifier, a bounding box, and a polygon defining its segmentation mask. The model outputs class probabilities, bounding boxes, confidence scores, and binary instance masks.

YOLO26 adopts an end-to-end architecture with a simplified prediction head that removes the conventional Non-Maximum Suppression stage and the Distribution Focal Loss branch. It also includes training and label-assignment mechanisms designed to improve localization, particularly for small or thin objects such as surgical instruments. In this work, the pretrained **YOLO26n-seg** model was fine-tuned on the customized endoscopic dataset. To ensure a fair comparison with YOLO11, both models were trained using the same dataset partitions and experimental settings: 640×640 input images, batch size 16, a maximum of 150 epochs, and an early-stopping patience of 100 epochs. This configuration provides a controlled basis for comparing the two YOLO architectures under the same experimental conditions.

The fused YOLO26n-seg model contains 139 layers, 2,692,979 parameters, and approximately 9.1 GFLOPs. When an input image is processed, the predicted tool instances can be combined into a single binary tool mask:

$$M_{\text{tool}}(x, y) = \begin{cases} 255, & \text{if at least one predicted tool instance covers } (x, y), \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

This aggregation is used to evaluate the overall spatial coverage of the detected instruments, although it discards instance identity and class-specific information.

2.4 Depth Anything V2

Depth Anything V2 estimates a monocular depth map from the RGB image. Since the output is relative rather than metric, it is normalized robustly using the first and ninety-ninth percentiles of finite values:

$$d_{norm} = \frac{\text{clip}(d, p_1, p_{99}) - p_1}{p_{99} - p_1}. \tag{6}$$

If the map is degenerate or contains no finite values, a zero map is returned. The depth map is resized to match the segmentation mask resolution using bilinear interpolation.

2.5 End-to-end inference

```
project_directory/
|-- Baseline+Yolo26.ipynb
|-- Baseline+Yolo11.ipynb
|-- ROImodel.pt
```

After fine-tuning YOLO26 and YOLO11 on the customized dataset (see *YOLO26 segmentation pipeline* section for more details), the two models are integrated into the baseline TTI detection pipeline to evaluate their downstream behavior under identical conditions. The baseline implementation is taken from the repository specified in [4], with the end-to-end inference procedure implemented in `Common Code/pipeline.py`.

For each input frame the inference procedure is organized as follows:

- **YOLO segmentation:** the input image is passed to the selected YOLO segmentation model, which returns a class label, confidence score, bounding box, and binary instance mask for each detected object. Classes 0–11 are treated as surgical-tool categories, with `tool_10_UNUSED` retained only for consistency with the dataset label mapping. Classes 12–20 are treated as interaction-related or tissue-related categories.
- **Depth estimation:** to provide additional spatial information a monocular depth map is computed from the original image using the `depth-anything/Depth-Anything-V2-Small-hf` model.
- **Tool–tissue pairing:** detected objects are separated according to their class groups. Candidate pairs are generated by combining each detected surgical tool with each tti object. If N tools and M tissue-related objects are detected, the pipeline produces up to $N \times M$ candidate pairs.
- **ROI extraction:** for each candidate pair, the corresponding tool and tissue-related masks are combined using a union operation. The smallest axis-aligned bounding box containing the union mask is then used to crop the corresponding region from the original RGB image and the depth map.
- **Multi-channel ROI construction:** the cropped RGB image is augmented with the cropped depth map and the combined tool–tissue mask. Each ROI is therefore represented by five channels: three RGB channels, one depth channel, and one binary union-mask channel.
- **Preprocessing:** the ROI is converted from an $H \times W \times 5$ array to a $5 \times H \times W$ PyTorch tensor, normalized by dividing the pixel values by 255, and resized to 224×224 pixels.
- **TTI classification:** the resulting five-channel tensor is passed to the `ROIClassifier`. This model is based on a ResNet-18 architecture extended with a learnable projection that maps the five-channel input to the three-channel representation expected by the original backbone. The trained weights are loaded from `ROImodel.pt`. The classifier then predicts the TTI class associated with the candidate tool–tissue pair (not the whole image) and a confidence score.
- **Prediction score:** the classifier logits are converted into class probabilities through a softmax operation. The predicted TTI class is selected according to the largest logit, and the corresponding maximum softmax probability is reported as the prediction score.

The complete inference process can be summarized as:

Input frame $\rightarrow$ *YOLO26n-seg or YOLO11n-seg* $\rightarrow$ *tool–tissue candidate pairs* $\rightarrow$ *RGB + depth + mask ROI* $\rightarrow$ *ROIClassifier* $\rightarrow$ *TTI prediction.*

3 Experimental Results

3.1 U-Net training behavior

The multiclass U-Net was trained independently from the YOLO-based models. It receives the complete RGB image and produces a pixel-wise semantic segmentation map over background, surgical-tool, and tool–tissue interaction categories.

The training log shows a consistent reduction in training loss and progressive improvement in foreground segmentation metrics. In one recorded run, the training F1 score increased from 0.0547 at epoch 1 to 0.5956 by epoch 21. The validation behavior was less stable. Although overall validation accuracy reached approximately 0.90 in some epochs, the validation foreground F1 score remained substantially lower, reaching approximately 0.19.

This discrepancy is expected in a highly imbalanced pixel-wise segmentation task. Background pixels occupy most of the image area, so a model can achieve high overall pixel accuracy even when predictions for surgical tools and interaction-related regions remain limited. Therefore, overall accuracy alone is insufficient to characterize U-Net performance.

Foreground-sensitive metrics provide a more informative evaluation, including macro precision, recall, F1, per-class Dice, per-class IoU, and the confusion matrix. Macro-averaged metrics reduce the dominance of the background class by assigning equal importance to each semantic category.

The gap between training and validation foreground metrics suggests overfitting to frequent visual patterns, particularly for rare tool and interaction classes with limited informative pixels. Consequently, U-Net results should be interpreted class by class rather than through aggregate accuracy.

Figure 1 shows a qualitative example of the U-Net prediction. The model identifies the dominant surgical instrument entering from the right side of the frame and traces its shaft and distal region, although a few small isolated false-positive components are present outside the main tool region.

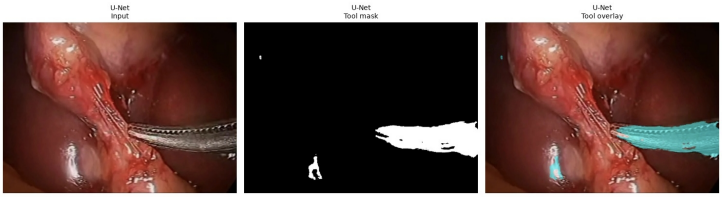

Figure 1: Qualitative U-Net segmentation result.

This example is consistent with the quantitative observations and highlights the importance of foreground-sensitive metrics. U-Net was evaluated as an alternative to YOLO26 but was not selected for the final end-to-end pipeline due to inferior segmentation performance.

3.2 YOLO26 results

The YOLO26n-seg model was evaluated using the best training checkpoint:

`/outputs/yolo26/train_yolo26/weights/best.pt`

Validation was performed on 760 images containing 2,041 annotated instances. The fused model has 139 layers, 2.69M parameters, and requires approximately 9.1 GFLOPs per forward pass.

Overall results are reported in Table 2. The model achieved box precision 0.616, recall 0.467, mAP@50 0.455, and mAP@50–95 0.367. For instance masks, the corresponding values were 0.601, 0.449, 0.426, and 0.292.

Table 2: Overall YOLO26n-seg validation performance on 760 images and 2,041 annotated instances.

Output	Precision	Recall	mAP@50	mAP@50–95
Bounding boxes	0.616	0.467	0.455	0.367
Instance masks	0.601	0.449	0.426	0.292

Performance varied substantially across classes (Table 3). The most reliably detected tool classes were `tool_1`, `tool_6`, and `tool_9`, with mask mAP@50 of 0.899, 0.805, and 0.925, respectively. In contrast, several rare tool classes and interaction-related categories showed considerably weaker performance.

Table 3: Selected class-wise YOLO26n-seg validation results.

Class	Instances	Box P	Box R	Box mAP@50	Mask P	Mask R	Mask mAP@50	Mask mAP@50–95
`tool_0`	18	0.000	0.000	0.000	0.000	0.000	0.000	0.000
`tool_1`	139	0.842	0.885	0.897	0.851	0.892	0.899	0.596
`tool_2`	8	0.686	0.750	0.745	0.686	0.750	0.745	0.542
`tool_3`	36	0.609	0.583	0.534	0.582	0.556	0.492	0.388
`tool_5`	14	0.717	0.905	0.816	0.717	0.904	0.816	0.593
`tool_6`	476	0.825	0.908	0.892	0.782	0.860	0.805	0.455
`tool_7`	4	0.580	1.000	0.995	0.582	1.000	0.995	0.771
`tool_8`	44	0.245	0.295	0.215	0.246	0.295	0.208	0.159
`tool_9`	406	0.879	0.958	0.930	0.877	0.956	0.925	0.764
`tool_11`	12	0.670	0.583	0.668	0.671	0.583	0.668	0.480
Unknown TTI	11	1.000	0.000	0.004	1.000	0.000	0.004	0.001
Other	46	0.000	0.000	0.007	0.000	0.000	0.009	0.002
Retract and grab	463	0.630	0.767	0.717	0.459	0.555	0.375	0.118
Blunt dissection	213	0.407	0.160	0.169	0.344	0.131	0.137	0.045
Energy-sharp dissection	148	0.378	0.139	0.152	0.421	0.148	0.166	0.047
Retract and push	2	1.000	0.000	0.000	1.000	0.000	0.000	0.000
Cut-sharp dissection	1	1.000	0.000	0.000	1.000	0.000	0.000	0.000

Training losses decreased over 150 epochs, while validation metrics improved during the initial phase and later fluctuated around a stable range (Figure 2).

Training losses decrease over 150 epochs, while validation metrics improve early and then fluctuate around a stable range. The model learns useful representations, particularly for frequent classes, although additional training does not yield uniform improvements across all labels. The divergence between training and validation is also compatible with overfitting, especially for rare classes.

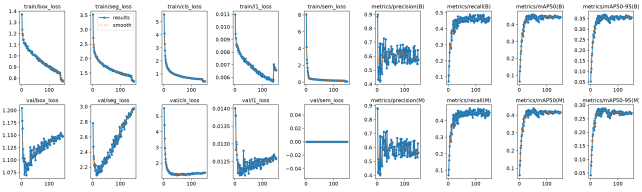

Figure 2: YOLO26n-seg training and validation curves over 150 epochs.

Validation required approximately 0.5 ms per image for preprocessing, 2.0 ms for inference, and 0.3 ms for postprocessing. Figure 3 shows a qualitative example in which YOLO26 predicts a single `tool_1` instance with confidence 0.64.

Validation required approximately 0.5 ms per image for preprocessing, 2.0 ms for inference, and 0.3 ms for postprocessing. In the qualitative example, YOLO26 predicts a single `tool_1` instance with confidence 0.64. The predicted mask isolates the main tool region and follows the visible instrument shaft, as confirmed by the overlay.

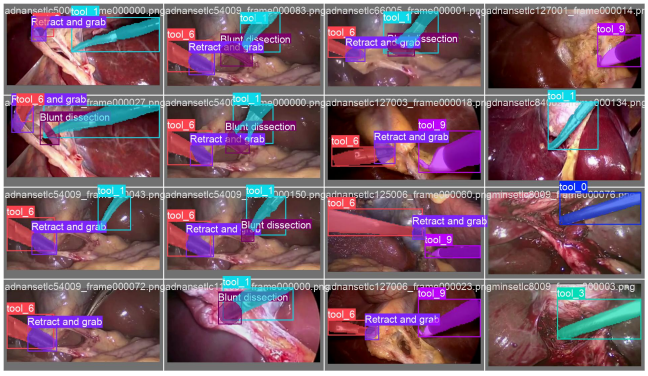

Figure 3: Qualitative YOLO26n-seg prediction.

The confusion matrix (Figure 4) confirms the class-wise trends observed in the metrics. Most errors involve rare tool and interaction classes, which are frequently confused with the background or with visually similar instrument categories.

Frequent tool categories exhibit stronger diagonal responses, consistent with their higher precision, recall, and mAP. Rare tool and interaction-related categories show weaker diagonal responses and greater confusion with the background or other classes. For example, `tool_0`, `tool_8`, and several TTI categories are frequently misclassified as background, while visually similar instruments (e.g., `tool_1` and `tool_9`) show higher mutual confusion.

Figure 4: YOLO26n-seg validation confusion matrix.

3.3 Evaluation metrics

For binary surgical-tool masks, the project computes intersection over union, Dice score, precision, and recall:

$$IoU = \frac{|P \cap G|}{|P \cup G|}, \tag{7}$$

$$Dice = \frac{2|P \cap G|}{|P| + |G|}, \tag{8}$$

where P is the predicted binary mask and G is the ground-truth mask. Precision and recall are computed from true positives, false positives, and false negatives.

For multiclass U-Net predictions, the evaluation includes pixel-wise accuracy, balanced accuracy, macro precision, recall, F1, per-class Dice, per-class IoU, and a confusion matrix. Macro-averaged metrics give equal weight to rare and frequent classes, avoiding dominance by the background class.

The depth-based contact-mask analysis is reported separately from tool-segmentation metrics. Since no ground-truth contact mask is available, the depth-based output is assessed through

qualitative visualizations and area-coverage measurements rather than supervised segmentation metrics.

Additional end-to-end quantities include wrong tool–tissue pairings and missed positive interactions. These measures evaluate the complete pipeline's ability to identify correct tool–tissue relationships, whereas tool-mask metrics assess pixel-wise segmentation quality.

3.4 End-to-End YOLO11 vs YOLO26 Comparison

The final evaluation stage compares YOLO11 and YOLO26 within the complete TTI detection pipeline described in Section 2.6. In both cases, the YOLO segmentation output is processed using the same Depth Anything V2 model and ROIClassifier, allowing the effect of the YOLO front-end on frame-level TTI prediction to be evaluated.

Two comparisons were performed. The first was a preliminary experiment between the official YOLO11 baseline and the YOLO26 model trained in this project. Since these models were not trained under identical settings, this comparison serves as an initial reference and is not used to draw conclusions about architectural differences.

At frame level, a sample is positive when at least one TTI annotation is present in the ground-truth label file. The pipeline predicts a positive frame when at least one candidate ROI is classified as a positive TTI by the ROIClassifier.

Table 4: Preliminary end-to-end comparison between the official YOLO11 baseline and the trained YOLO26 model.

Metric	**YOLO11 pipeline**	**YOLO26 pipeline**
Accuracy	0.929	0.720
Precision	0.958	0.918
Recall	0.951	0.706
F1-score	0.954	0.798
Specificity	0.846	0.769

In this preliminary experiment, the official YOLO11 baseline achieves higher performance across all metrics. The largest difference is in recall, which decreases from 0.951 for YOLO11 to 0.706 for YOLO26. Confusion counts are reported in Table 5.

Table 5: Confusion counts for the preliminary end-to-end comparison.

Outcome	**YOLO11 pipeline**	**YOLO26 pipeline**
True Positive (TP)	136	101
True Negative (TN)	33	30
False Positive (FP)	6	9
False Negative (FN)	7	42

The larger number of false negatives produced by YOLO26 indicates that this pipeline misses more frames with actual tool–tissue interactions. However, because the models were trained under different conditions, the observed difference cannot be attributed solely to architecture.

3.4.1 Controlled Final Comparison

To obtain a fairer comparison, a second experiment was performed in which YOLO11 and YOLO26 were trained using identical settings. The same test set, Depth Anything V2 model, ROIClassifier, and evaluation protocol were used for both pipelines.

Results are reported in Table 6.

Confusion counts are reported in Table 7.

Table 6: Controlled end-to-end comparison between YOLO11 and YOLO26.

Metric	**YOLO11 pipeline**	**YOLO26 pipeline**
Accuracy	0.802	0.681
Precision	0.896	0.938
Recall	0.846	0.636
F1-score	0.871	0.758
Specificity	0.641	0.846

Table 7: Confusion counts for the controlled end-to-end comparison.

Outcome	**YOLO11 pipeline**	**YOLO26 pipeline**
True Positive (TP)	121	91
True Negative (TN)	25	33
False Positive (FP)	14	6
False Negative (FN)	22	52

YOLO11 provides better overall performance, achieving higher accuracy (0.802), recall (0.846), and F1-score (0.871), whereas YOLO26 achieves higher precision (0.938) and specificity (0.846).

YOLO26 produces fewer false positives than YOLO11 (6 versus 14), but substantially more false negatives (52 versus 22). Therefore, YOLO26 behaves more conservatively: when it predicts a positive interaction, the prediction is more frequently correct, but it misses more frames with actual tool–tissue interactions.

YOLO11 provides the better trade-off for the end-to-end TTI detection task, primarily because of its substantially higher recall and F1-score. Since both models were trained and evaluated under identical conditions, this controlled comparison provides the primary basis for the final comparison between YOLO11 and YOLO26.

4 Discussion and Findings

The experiments reveal four main findings. First, dataset alignment is a prerequisite for meaningful model comparison. The identical split counts and absence of unmatched image stems indicate that YOLO26 and U-Net are evaluated on corresponding samples, allowing observed differences to be attributed to model architecture, data representation, and inference procedure rather than to data partitioning.

Second, YOLO26 and U-Net provide different practical trade-offs. YOLO26 predicts object instances, bounding boxes, class labels, and segmentation masks, making it suitable for multi-instance tool localization and integration into the end-to-end TTI pipeline. On the validation set, YOLO26 achieved box mAP@50 of 0.455 and mask mAP@50 of 0.426, with substantial class-wise differences: `tool_1`, `tool_6`, and `tool_9` were segmented relatively well, whereas rare tool and interaction-related categories showed weaker performance. This variability is associated with the highly uneven instance distribution: classes with hundreds of instances (`tool_6`, `tool_9`, *Retract and grab*) are learned more effectively than those with few examples (`tool_7`, *Retract and push*, *Cut-sharp dissection*). However, instance count is not the only factor: `tool_8` achieves mask mAP@50 of only 0.208 despite 44 instances, while `tool_7` reaches 0.995 with only four instances, so its results should be interpreted cautiously. Interaction-related classes are more difficult to segment, with *Retract and grab* achieving mask mAP@50–95 of 0.118 and *Blunt dissection* and *Energy-sharp dissection* reaching only 0.0445 and 0.0465.

U-Net provides a complementary semantic-segmentation perspective, assigning one class per pixel. This representation is useful for dense analysis and generating binary tool and TTI masks, but does not preserve individual object identities. U-Net results are strongly affected by class

imbalance and background dominance, so performance should be interpreted using foreground-sensitive metrics such as macro precision, recall, F1, per-class Dice, and IoU.

Third, depth fusion is a useful but indirect approach to contact estimation. The method assumes that a relative-depth subset of the predicted tool corresponds to the contact-relevant part, an assumption that can fail with ambiguous depth ordering, occlusions, or artifacts around specular highlights. Percentile normalization reduces the influence of extreme values, but values cannot be compared across frames as physical distances. The quantile rule should be interpreted as a rank-based spatial filter rather than a metric contact measurement. The selected quantile $q = 0.25$ yields a selective region; increasing q expands the candidate region, whereas decreasing q makes selection more conservative. These parameters should be tuned on validation and kept fixed for test evaluation.

Training curves show YOLO26 losses decreasing throughout 150 epochs, while validation metrics improve early and later fluctuate, compatible with overfitting for rare classes. The comparison highlights the importance of separating model metrics from pipeline metrics: YOLO26 box and mask mAP measure segmentation quality, whereas contact coverage, incorrect pairings, missed interactions, and TTI scores measure the complete system.

Fourth, the controlled end-to-end comparison shows that YOLO11 achieved better overall performance, particularly in recall and F1-score, whereas YOLO26 showed higher precision and specificity. YOLO26 behaved more conservatively, producing fewer false positives but substantially more false negatives, making YOLO11 the better trade-off for the end-to-end TTI detection pipeline.

5 Conclusions

This project develops an end-to-end framework for surgical-tool perception in endoscopic images. YOLO26 and U-Net provide complementary segmentation representations: YOLO26 produces object-level instances, whereas U-Net produces dense semantic masks. Depth Anything V2 is used as an auxiliary relative-depth module for extracting contact-region hypotheses from predicted tool masks. Dataset preparation is reproducible and consistent, with 2,880 training, 760 validation, and 182 test frames.

The best YOLO26n-seg checkpoint achieved box precision 0.616, recall 0.467, mAP@50 0.455, and mAP@50–95 0.367. For instance masks, corresponding values were 0.601, 0.449, 0.426, and 0.292 on 760 validation images with 2,041 annotated instances. These metrics should be interpreted together with class-wise results: frequent tool classes (`tool_1`, `tool_6`, `tool_9`) were segmented relatively well, whereas rare tool and interaction-related categories showed weak or unstable performance.

Validation results reveal a highly uneven class distribution. Classes with several hundred instances generally provide more reliable estimates and are learned more effectively than classes with few examples. However, instance count is not the only performance determinant, since visual ambiguity, occlusion, appearance, and annotation complexity also affect results. Future evaluations should report aggregate metrics together with class-wise metrics and validation instance counts.

Depth-based fusion produces interpretable contact-mask candidates using information beyond the binary tool mask. However, it is a heuristic based on relative monocular depth rather than a direct physical contact measurement. Parameters and selection direction must be validated against interaction annotations. The final system should report segmentation, contact, TTI classification, and computational performance as separate but connected evaluation categories.

Finally, the controlled end-to-end comparison showed that YOLO11 achieved the best overall trade-off for TTI detection, with higher accuracy, recall, and F1-score. YOLO26 achieved higher precision and specificity but produced substantially more false negatives.

References

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention*, 2015.

[2] L. Yang et al., "Depth Anything V2," 2024. Project implementation uses a Hugging Face depth-estimation pipeline with a Depth Anything V2 checkpoint.

[3] G. Jocher et al., "Ultralytics YOLO: Unified real-time end-to-end vision models," project documentation and implementation used for instance segmentation.

[4] Repository given together with the project. The main folder used is Common Code. https://github.com/Carlo1109/TTI_Detection.git

[5] Medical Robotics Project, "Endoscopic tool-segmentation dataset exported in YOLO and U-Net representations," project files and evaluation notebooks.